

VITYARTHI PROJECT

STOCK PORTFOLIO SIMULATOR

A Python-based educational market simulation

PREPARED BY

Aryan Kamlakar Gore

CSE in AI/ML

VIT Bhopal

ACADEMIC YEAR

2026

Academic Project Report

1. Introduction

Stock Portfolio Simulator is a terminal-based Python application for exploring how trading decisions, portfolio allocation, and synthetic market conditions affect value and risk. It uses a fixed set of fictional stock records and virtual cash; it does not connect to a broker, fetch live prices, or execute real-money trades.

The intended users are students, programming learners, and instructors who need a transparent example of modular Python design. A user can inspect a fictional market, buy or sell shares, advance a seeded stochastic market, view portfolio metrics, compare volatility scenarios, generate a text report, and save or reload state.

```

rnmmaximus@rnmmaximus:~/Desktop/vityrathi$ python main.py

=====
      STOCK PORTFOLIO SIMULATOR
=====
1. View market
2. View portfolio
3. Buy stock
4. Sell stock
5. View transactions
6. Run market simulation
7. Analyze portfolio
8. Compare scenarios
9. Generate report
10. Save simulation
11. Load simulation
0. Exit

Choose an option: 1

Symbol  Name                Sector      Price      Change
-----
INFY    Infosys Ltd.         Technology  ₹ 1014.00   0.00%
NVDA    Nvidia               Technology  ₹22357.00   0.00%
CIPLA   Cipla                Healthcare  ₹ 1343.00   0.00%
ADNPW   Adani Power Ltd.     Energy      ₹ 199.00    0.00%
HDFC    HDFC Bank Ltd        Finance     ₹ 728.00    0.00%

```

Figure 1: Interactive simulator menu and market view captured from the terminal.

2. Problem Statement

Manually reasoning about a portfolio across changing prices is repetitive and makes it difficult to reproduce the same experiment. Learners also need a safe environment in which invalid trades, insufficient cash, changing prices, and risk measures can be observed without financial consequences.

The application addresses this problem with a small, inspectable model: fictional stocks are held in a Market, portfolio state is represented by dataclasses, TradingEngine validates orders, Simulator advances prices with random.Random, and Analysis derives return, drawdown, and volatility from the resulting equity curve.

3. Functional Requirements

ID	Functional requirement	Implemented behavior
FR-01	View market	Displays symbol, name, sector, price, and latest percentage change for the default five-stock market.
FR-02	Manage trades	Buys and sells integer shares after checking symbol, share count, cash, and holdings; applies the configured fee rate.
FR-03	Track portfolio	Maintains cash, positions, average cost, transactions, and calculated equity.

ID	Functional requirement	Implemented behavior
FR-04	Simulate prices	Advances the market for a requested number of days using seeded Gaussian changes, drift, and volatility.
FR-05	Analyze results	Calculates return, maximum drawdown, and population volatility from an equity curve.
FR-06	Compare scenarios	Runs Low, Medium, and High volatility examples and prints their summary metrics.
FR-07	Persist state	Saves and loads market and portfolio state as JSON.
FR-08	Export curve	Writes day/equity rows to CSV; an optional Matplotlib chart can be saved.
FR-09	Interact in two modes	Supports an interactive menu when no arguments are supplied and an argparse demo mode otherwise.

4. Non-Functional Requirements

Quality	Evidence in the implementation	Boundary
Usability	Readable terminal menu, formatted currency, descriptive operation errors.	Interactive input remains terminal-only.
Reliability	ValidationError, ValueError, OSError, KeyError, and TypeError are handled at the menu boundary.	Malformed JSON can still fail during direct library use.
Maintainability	Focused packages for models, market, trading, simulation, analysis, storage, and reports.	Some functions use compact formatting and minimal docstrings.
Portability	Core behavior uses the standard library; optional plotting is guarded.	Matplotlib is required only for plotting.
Reproducibility	Simulator accepts a seed and owns a random.Random instance.	Random outcomes still depend on model parameters.
Performance	In-memory lists/dictionaries and linear scans suit the small dataset.	No large-scale benchmark or optimization is claimed.

5. System Architecture

Architecture and data flow

User → **main.py** → **modules.main** → interactive menu or argparse demo
 Interactive/demo control → **Simulator** → **Market** (Stock records) + **TradingEngine**
TradingEngine → **Portfolio** (Position and Transaction records)
Simulator → **Analysis** → formatted report / optional plot
 Save/load → **JsonStore** → JSON file; export → **export_equity** → CSV file

Architecture and data flow (conceptual representation derived from the implementation).

The repository-level main.py is only a launcher. The application logic is in modules/main.py, which creates a Simulator, routes user choices, and delegates domain work to focused packages. State is primarily in memory; JSON and CSV are explicit file boundaries rather than a database layer.

6. Design Diagrams

6.1 Use-case view

User → View market | View portfolio | Buy stock | Sell stock

User → View transactions | Run market simulation | Analyze portfolio

User → Compare scenarios | Generate report | Save simulation | Load simulation

6.1 Use-case view (conceptual representation derived from the implementation).

6.2 Workflow view

Start → inspect command-line arguments

No arguments → display menu → read choice → validate → perform operation → continue or exit

Arguments → create seeded Simulator → run requested days → report / CSV / plot / JSON → exit

6.2 Workflow view (conceptual representation derived from the implementation).

6.3 Sequence view: a simulation and analysis

```

User → modules.main: request simulation

modules.main → Simulator.run(days)

Simulator → Market.step(random generator, volatility)

Simulator → TradingEngine.value(Market) → equity curve

modules.main → Analysis.summary(curve) → return, drawdown, volatility

modules.main → User: formatted result

```

6.3 Sequence view: a simulation and analysis (conceptual representation derived from the implementation).

6.4 Class diagram

```

Market composes Stock records and exposes quote(), step(), snapshot()

Portfolio composes Position and Transaction records and calculates equity()

TradingEngine uses Market and Portfolio to implement buy()/sell()

Simulator owns Market, TradingEngine, and Random; returns SimulationResult

Analysis is a stateless utility for summary() and compare()

```

6.4 Class diagram (conceptual representation derived from the implementation).

6.5 ER diagram: Not Applicable. The current implementation does not use SQLite, another relational database, or persistent entity-based storage. JSON contains serialized market and portfolio dictionaries, while CSV contains an exported equity curve; these are file formats, not an ER model.

7. Design Decisions & Rationale

Decision	Reason	Trade-off
Use dataclasses for domain records	Stock, Position, Transaction, Portfolio, and SimulationResult have clear state and straightforward serialization.	Validation and business rules still need explicit methods.
Separate packages by responsibility	Market, trading, simulation, analysis, storage, and reporting can be read and changed independently.	More files and imports than a single-script prototype.
Use dictionaries keyed by symbol	Fast direct lookup makes quotes and positions simple for a small fixed market.	Ordering and relational queries are not the focus.
Use seeded Random	A seed makes demonstrations and experiments repeatable while retaining stochastic behavior.	The model is synthetic and not a market forecast.
Use JSON/CSV files	Human-readable persistence and simple analysis export without a database dependency.	No concurrent access, schema migration, or query support.
Keep plotting optional	Core simulator remains standard-library based and usable in a terminal.	Chart generation is skipped when Matplotlib is unavailable.

8. Implementation Details

models/core.py — domain model

Purpose: Defines Stock, Position, Transaction, Portfolio, and ValidationError. **Input:** symbols, prices, shares, fees, and serialized dictionaries. **Processing:** normalizes symbols, validates positive values, updates histories, computes transaction totals and portfolio equity. **Output:** typed objects and dictionaries through `to_dict()/from_dict()`.

market/market.py — market state

Purpose: Stores Stock objects in `Market.stocks` and tracks day. **Functions:** `add_stock()`, `quote()`, `step()`, `snapshot()`, `default_market()`. **Processing:** each step applies drift plus a Gaussian draw to each stock, rounds the new price, and appends history. **Output:** quotes, snapshots, and serialized market state.

trading/engine.py — trade rules

Purpose: Centralizes buy and sell operations. **Processing:** converts share input to int, looks up price, computes fees, checks cash or holdings, updates positions/cash, and appends a timestamped Transaction. **Output:** a completed transaction or ValidationError.

simulation/simulator.py — experiment loop

Purpose: Runs multi-day experiments. **Functions:** `Simulator.run()` and `SimulationResult.to_dict()`. **Processing:** records initial equity, advances Market, optionally calls a strategy, records daily equity/prices, and returns final cash and transactions.

analysis/analyzer.py — metrics

Purpose: Provides stateless summary and compare functions. **Processing:** iterates over adjacent equity values, tracks peak-to-trough drawdown, computes simple returns and population standard deviation. **Output:** dictionaries containing initial, final, `return_pct`, `max_drawdown_pct`, and `volatility_pct`.

storage/json_store.py and csv_store.py — files

Purpose: `JsonStore.save/load` serializes market and portfolio state; `export_equity` writes day/equity rows. **Output:** JSON and CSV files through `pathlib`. No database or network service is used.

reports/report.py and modules/main.py — presentation

Purpose: `text_report` formats metrics and `plot_equity` optionally uses Matplotlib. `modules.main` handles argparse, the menu loop, display helpers, input conversion, and error reporting. **Output:** terminal text, optional PNG, JSON, and CSV artifacts.

```
new_price = old_price * (1 + drift + random_gaussian_change) # implemented by Market.step()
with rng.gauss(0, volatility)
```

The implementation demonstrates variables, floats and integers, lists and dictionaries, loops, conditionals, functions, modules/packages, dataclasses, properties, exceptions, JSON/CSV file handling, and composition between domain objects. It does not use inheritance, a web framework, authentication, a database, or machine learning.

9. Screenshots / Results

The supplied captures are terminal evidence of the interactive workflow. Their aspect ratios are preserved; they are resized only to fit the report frame.

```

=====
      STOCK PORTFOLIO SIMULATOR
=====
1. View market
2. View portfolio
3. Buy stock
4. Sell stock
5. View transactions
6. Run market simulation
7. Analyze portfolio
8. Compare scenarios
9. Generate report
10. Save simulation
11. Load simulation
0. Exit

Choose an option: 3
Stock symbol: INFY
Number of shares: 5
Buy order completed: 5 INFY at ₹1014.00.

```

Figure 2: Market table showing symbols, sectors, prices, and percentage changes.

```

=====
      STOCK PORTFOLIO SIMULATOR
=====
1. View market
2. View portfolio
3. Buy stock
4. Sell stock
5. View transactions
6. Run market simulation
7. Analyze portfolio
8. Compare scenarios
9. Generate report
10. Save simulation
11. Load simulation
0. Exit

Choose an option: 2

Cash: ₹3,729.74
Total value: ₹9,993.74

Symbol  Shares  Average cost  Market value
INFY    5      ₹    1015.01  ₹    5070.00
ADNPW   6      ₹    199.20  ₹    1194.00

```

Figure 3: Terminal interaction capture supplied with the project (preserved at source aspect ratio).

```

Choose an option: 6
Simulation days: 30
Advanced to market day 30. Portfolio value: ₹10,663.71

=====
      STOCK PORTFOLIO SIMULATOR
=====
1. View market
2. View portfolio
3. Buy stock
4. Sell stock
5. View transactions
6. Run market simulation
7. Analyze portfolio
8. Compare scenarios
9. Generate report
10. Save simulation
11. Load simulation
0. Exit

Choose an option: 7
Simulation report
=====
Initial equity: 9993.74
Final equity: 10663.71
Return: 6.70%
Max drawdown: 2.69%
Volatility: 1.02%
Trades: 2
Volatility: 1.02%

```

Figure 4: Terminal interaction capture supplied with the project (preserved at source aspect ratio).

10. Testing Approach

No test modules are present in the repository at the time of report generation. pytest.ini configures the repository root on Python's import path, and the README describes behavior-focused tests, but no test files were available to report as executed automated tests. The cases below are therefore recommended/manual verification cases derived from the implemented behavior; they are not claimed as committed test results.

ID	Test case	Input	Expected result	Status
T-01	View default market	Interactive option 1	Five fictional stocks are displayed with price/change columns.	Manual scenario
T-02	Valid buy	Known symbol and affordable positive shares	Cash decreases, position and transaction are created.	Recommended
T-03	Invalid buy	Unknown symbol or insufficient cash	Validation message is printed; program continues.	Recommended
T-04	Invalid sell	More shares than held	Validation message is printed; holdings remain valid.	Recommended
T-05	Zero-day simulation	demo --days 0	A result contains initial equity and no market advance.	Recommended
T-06	Seed repeatability	Same --days and --seed twice	The generated price path is reproducible.	Recommended
T-07	Persistence	Save then load JSON	Market day, prices, cash, positions, and transactions are restored.	Recommended
T-08	CSV export	demo --csv equity.csv	CSV has day/equity header and one row per curve value.	Recommended

11. Challenges Faced

Input and trade validation

Problem: User input is text and trades can violate cash/holding constraints. **Approach/result:** The menu converts integer input and catches `ValueError/ValidationError`; `TradingEngine` checks cash and holdings. Direct callers still need to handle exceptions.

Reproducible stochastic behavior

Problem: Random price changes should vary while demonstrations should be repeatable. **Approach/result:** Simulator owns `random.Random(seed)` and passes it to `Market.step()`. The model remains synthetic rather than financially predictive.

Accounting across changing prices

Problem: Portfolio value combines cash with current market values while average cost and fees must remain coherent. **Approach/result:** `Portfolio.equity()` queries current prices and `TradingEngine` updates positions/transactions. Tax, slippage, and realized P/L are outside scope.

Keeping the core lightweight

Problem: Charts are useful but should not make the basic terminal program depend on a plotting stack. **Approach/result:** `Matplotlib` is imported only inside `plot_equity()`; plotting returns `False` when unavailable.

12. Learnings & Key Takeaways

- Dataclasses make related state explicit and support readable serialization boundaries.
- Dictionaries keyed by stock symbol provide a practical lookup model for quotes and positions.
- Small functions and packages make a terminal application easier to inspect than a monolithic script.
- Exception handling turns invalid user actions into recoverable terminal messages.
- A seeded random generator supports repeatable experiments and makes debugging stochastic code easier.
- Equity curves turn daily portfolio values into measurable return, drawdown, and volatility summaries.
- JSON and CSV demonstrate file handling without introducing a database dependency.

13. Future Enhancements

Horizon	Possible improvement (not currently implemented)
Short-term	Add repository tests, strengthen numeric/input validation, improve report/menu formatting, and document realized profit/loss and fee behavior.
Medium-term	Add configurable portfolios, richer order types, slippage, tax/commission models, more analytics, and persistent experiment history.
Long-term	Add a GUI or web interface, database-backed experiment storage, live-data adapters behind an explicit boundary, and advanced portfolio strategies.

14. References

[1] Python Software Foundation. *Python 3 Documentation*. <https://docs.python.org/3/> (accessed 24 September 2026).

[2] Python Software Foundation. *dataclasses — Data Classes*. <https://docs.python.org/3/library/dataclasses.html> (accessed 24 September 2026).

[3] Python Software Foundation. *random* — Generate pseudo-random numbers. <https://docs.python.org/3/library/random.html> (accessed 24 September 2026).

[4] Matplotlib development team. *Matplotlib documentation*. <https://matplotlib.org/stable/> (optional plotting dependency used by reports/report.py; accessed 24 September 2026).

[5] Project source and README in this repository: Stock Portfolio Simulator, modules/ package, statement.md, and README.md.